Analyze the five Vs

	Example	Effect on the design
Volume	About 3 million tap records per day, which is around 1.1 billion per year and 5.5 billion over five years.	Scalable storage and adapt to new vehicles as they are added
Velocity	A ticket tap can take about 30 seconds per person. It can change depending on if it is during rush hours.	A design that won't create queue during rush hours
Variety	The data comes in different formats such as tap records, GPS event streams, video from road cameras, and more.	Storage that can process different formats of data and define the keys for each.
Veracity	Tap records can also be duplicated or contain invalid card IDs	Validation, de-duplication, and quality checks, so that alerts and reports are not based on wrong data.
Value	Counting taps per route and hour shows demand, which can be used for reports and for adjusting schedules.	Reporting with aggregated data, while passenger card IDs are anonymized and access is restricted by role.

B2

Limitations of one conventional database server

Not enough capacity for the data since in some years, together with videos and JSON logs will fill the one server's storage and slow the queries as well as refreshes.

Moreover, the server must get events every few seconds, raise alerts within 30 seconds, and also run reports. All these jobs compete for the same resources in one server.

If one process fails it can impact the whole process in the same server and can be hard to trace the error.

Scale-up and scale-out

Scale-up means making one server more powerful by adding CPU, RAM, or disk. It gets expensive quickly and still leaves one point of failure.

Scale-out instead means adding more servers that share the work and the data. It can grow by adding data and connections and the system keeps running if one machine fails, but it is complex to manage than one server.

I recommend a scale-out architecture with two paths: a path for alerts and a path for storage and reports.This system handles growth: adding buses means adding nodes, not replacing a server. A data lake accepts every format, including video and JSON, without forcing it into tables. Storage is cheap enough for five years, with older data moved to cold tiers.

B3

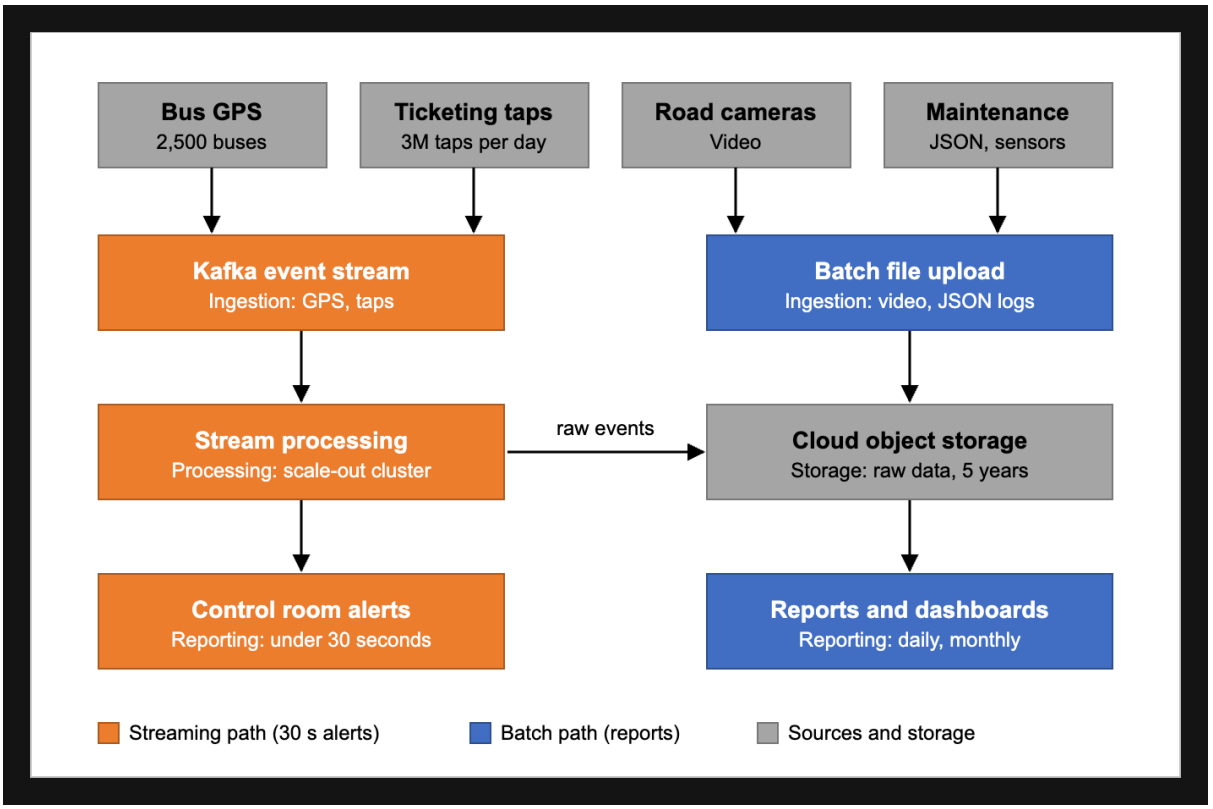
Workload	Classification	Reason
Delay alerts	Streaming	Must be raised within 30 seconds so each GPS event has to be processed as soon as it arrives.
Daily route reports	Batch	They summarize a full day of data and are only needed once the day is over, a scheduled job is enough.
Long-term planning	Hybrid	It uses years of historical data but it can also use recent live data to stay up to date.

Kafka would sit between the data sources and the systems that use the data. It receives events, stores them for a set time, and lets several consumers read them independently.

Video would normally go straight to object storage, with only its metadata passing through Kafka, because video files are too large for efficient use as Kafka messages.

Streaming is more complex and costly to run than batch, and most workloads don't need it. Daily and monthly reports are only read once the period is complete.

B4



GPS events from the buses and tap records from the ticketing system go into Kafka, which gets the continuous stream. A scale-out stream processing cluster reads these events, compares them with schedules, and sends delay and congestion alerts to the control room within 30 seconds. Kafka also sends a raw copy of every event to cloud object storage, where camera video and JSON maintenance logs also arrive through batch file upload. Storage and compute are separate, so each can grow independently as buses are added. Daily and monthly reports for planners read from the stored data.

B5

Combining the data sources can reveal where a specific person travels. A card ID in the tap records, joined to the bus ID and GPS location at the tap time, shows the stops and times a passenger uses, which can expose their home, workplace, or routine.

For this reason, planners get access only to aggregated, pseudonymized report data, and not to the raw tap records or raw GPS data. Object storage and Kafka keep the raw, timestamped events with source IDs, so any figure in a report can be traced back to the

records it came from. This supports both audits of who used sensitive data and checks of how a result was produced.